Machine Learning for SNAP Payment Accuracy: Project Summary

October 26, 2026 (corrected) · Project summary, September 28 – October 26 · Cory Trimm

> **Corrected October 26, 2026.** Earlier versions overstated the ranking tool's results. The models used the benefit the agency issued alongside income, deduction and household-size values that, in the federal QC file, are what the QC reviewer recorded, so they could partly see the error. Comparing with an independent project (giannella/snap_qc) exposed it (experiment 106). Every ranker and cost-share figure here comes from the October 26 rebuild: leak-free inputs (experiment 107), trained and scored on the errors that count toward the official payment error rate, errors the QC reviewer found whose amount is over the federal tolerance (experiments 111–112). The cost share uses the tool's lift on those dollars; on the earlier any-error basis it had read $3.42 billion (experiment 110), and on an interim label built from the benefit gap $3.15 billion (experiment 111). The synthetic-data, language-model and policy-question results did not use these models.

Executive summary

States lose money, and from FY2028 will pay a share of benefit costs, when SNAP benefits are paid in the wrong amount. Quality control (QC) reviewers can examine only a small share of cases, and caseworkers have to apply a large, changing body of policy correctly. Over four weeks this project ran 105 numbered experiments, plus pilot tracks for six states. It asked where machine learning can actually help. The data was five years of federal QC records (FY2020–2024, about 167,000 reviewed cases), a synthetic case generator, and a policy library built for the purpose.

The project produced six findings, in order of how ready they are to use:

1. **A ranking tool can roughly double or triple the error dollars a review finds, in every state.**
    - **What it does:** a conventional model, built for all 53 jurisdictions, ranks active cases by expected error dollars using only what the agency has on file before

review.

- **Accuracy:** on cases it never saw, the top 20% of its queue holds a median 3.0 times the error dollars of random selection, counting the errors the official rate counts (lowest: Kansas, 2.18 times), and the top 10% holds 4.1 times. Its dollar forecasts are within 20% of actual in 38 of 53 jurisdictions.
- **An independent check:** on giannella/snap_qc's one-year-ahead test, with both tools on their rebuilt pre-review inputs, it found more error dollars than their rule lists in 35 of 49 states (experiments 106 and 109).
- **Fairness:** a floor that keeps elderly and disabled households' share of reviews proportionate costs the median state about 4% of its lift (the most, Utah, 45%). It leaves 8 jurisdictions below 0.8 for elderly households.

2. **Under the FY2028 cost-sharing rules, that could be worth billions.**
    - **Exposure:** at FY2025 error rates, 43 jurisdictions would owe about $10.6 billion a year.
    - **Avoided:** in a central scenario, reviewing 2% of cases a month in ranker order (fairness floor on), states would avoid an estimated $4.47 billion of it a year, against $1.03 billion for random reviews of the same volume. The range across scenarios is $0.21–8.24 billion.
3. **Realistic synthetic cases teach real skills.** A generator calibrated to the federal data produces fake cases with known errors. Models trained only on these fake cases predict errors in real cases well above chance, reaching about half to two-thirds of what models trained on real data achieve.
4. **A small language model can learn part of a QC reviewer's job from synthetic data alone.**
    - **Result:** Gemma 4 E4B (about 4 billion parameters), fine-tuned only on synthetic cases, passed all three pre-registered checks on 1,200 fresh real cases. Its overall score was 33% (28% above chance), against 0.5% untrained and 24% for a conventional classifier.
    - **Limits:** it spots errors, their direction and where they are well above chance, but not the type of mistake or whose mistake it was.
5. **Plain-language flag cards can tell a reviewer where to look, with honest odds and correct citations.**

- **Where to look:** the cards' top three "check first" items include the real error for 83–89% of flagged errors in Pennsylvania and New York (exp108). A fixed list of each state's three most common error locations would get about 79%, so the model's value is mostly in which item it lists first.
- **Honest odds:** their odds were recalibrated so that "7 in 10" means about 7 in 10.
- **Citations:** a small model writes the card text, and its rule citations matched a much larger model's 100% of the time on 517 fresh cards.

6. **Looking up policy helps a mid-sized open model answer policy questions, up to a point.**
    - **Result:** on Cornell's 500-question CalFresh quiz, Gemma 4 31B went from 84.0% to 89.6% with lookup tools over a dated policy library. Nava PBC published 85.8% for the same model without tools.
    - **The plateau:** ten further days of changes, including a confirmed routing layer, held it at about 90%. Untuned GPT-6 scores 92.2% without tools (Nava's figure).
    - **Details:** a companion summary covers this track (*Grounding an Open Model in SNAP Policy*).

What's ready: the ranker (with the fairness floor) and the cost-share model. The flag cards are ready for a reviewer trial. The language models are research results: useful as helpers that explain and cite, not as decision makers.

Recommendation: pilot the ranker in one state (Pennsylvania or New York), with the fairness floor on and flag cards attached. Measure dollars found against the forecast, and run the blind reviewer comparison of the flag cards in the same pilot.

What we tried, in plain terms

Question we asked	What we built	What we found
Can we make realistic fake SNAP cases?	A generator: fake households, the federal benefit formula, and errors injected at the rates seen in real QC data	Yes. The mix of errors matches the real data to within about 2%, and the dollar error rate to within about 1 point
Do fake cases teach anything real?	Models trained only on fake cases, tested on real ones	Yes, every time; about half to two-thirds as well as training on real data. More fake data stops helping past about 10,000 cases; realism matters more
Can a small AI model read a case like a reviewer?	A 4-billion-parameter model trained on fake cases, tested on 1,200 real ones under rules set in advance	Partly. It finds that there's an error, which way and where; it can't yet say what kind or whose
Which cases should reviewers look at first?	A ranking model per state that estimates each case's likely error dollars	The top fifth of the queue holds about 3.0 times the error dollars of a random fifth (errors over the tolerance; median; 2.0 times or more in all 53)
Does ranking by dollars shortchange anyone?	A fairness audit of review shares by household type, and a floor	Elderly households were under-reviewed everywhere; a floor fixes it for a median 10% of the gain per state
What is it worth under the new cost-share rules?	A scenario model of error dollars in each state's caseload	About $3.4 billion a year avoided nationally in the central scenario ($0.2–6.6 billion range)
Can we tell reviewers *why* a case was flagged?	Flag cards: why it was flagged, what to check first, the rule that applies	The top three checks catch 83–89% of errors (a fixed list of the commonest gets about 79%); the odds are calibrated; citations are correct

Question we asked	What we built	What we found
Can an AI model answer policy questions correctly?	A searchable, dated library of federal and state SNAP policy, and tools to search and open it	A mid-sized model goes from 84% to about 90% on a public CalFresh quiz; a small one doesn't gain

How the pieces fit

The federal QC sample is random by law, and no tool should touch it. The opportunity is everything a state does **before** that sample is drawn: its own reviews of active cases.

Step	Who or what	What happens
1. Every month	**The ranker**	Scores every active case from information on file and estimates its expected error dollars
2. Choose cases	**The ranker**, with the fairness floor	Puts the cases most likely to hold large errors at the top of the review queue, keeping each household type's share proportionate
3. Review	**State reviewers**, with a **flag card** per case	The card says why the case was flagged, what to check first, and which rule applies
4. Policy questions	**Reviewers and caseworkers**, optionally with the **policy assistant**	Look up the rule in force on the case's date, with the citation
5. Fix causes	**State program staff**	Fix the policies, notices or system issues behind repeated errors
6. Federal QC	**Federal and state QC staff**	The usual random sample; errors already corrected are less likely to be found

The ranker carries steps 1 and 2, where the dollar gains come from. The language models have no role in choosing cases. Their role is in steps 3 and 4, explaining and citing.

Results

1. The ranker

All 53 jurisdictions, held-out cases	Result
Top 20% of the queue vs random review (errors over the tolerance)	median 3.0× (lowest Kansas 2.18×, highest Washington 3.90×)
Top 10% of the queue	median 4.1×
Train on 2020–22, test on 2023–24	median 3.0× (lowest Kansas 2.20×)
Error dollars found per review hour	$29–245; median $109 (Pennsylvania $109, against $36 at random)
Dollar forecast within 20% of actual	38 of 53
Cost of the fairness floor	median lift 3.0× → 2.8×; median state about 4%, most 45% (Utah)

How it works:

- **Two parts:** one estimates the chance a case has an error, the other the dollars if it does. Cases are ranked by the product.
- **Pooled training:** each jurisdiction's model learns from every state's QC cases plus its maximum allotments. Pooling beat state-only models in all 53 jurisdictions when it was adopted (exp80, exp82). Re-run without the leak it still does: 2.70× against 2.27×, better in 46 of 53 (exp108, any-error basis).
- **No issued benefit:** in the federal QC file the "reported" income, deduction and household-size fields are what the QC reviewer recorded. Pairing them with the issued benefit let the earlier models see the error (exp106), so the issued benefit is no longer an input (exp107). The reported facts remain, and for error cases they are the corrected ones; a pilot on a state's own pre-review records is the real test.

Underpayments: a second ranker trained on underpayments finds a median 1.8 times the underpayment dollars of random review, against 1.5 times for the main ranker, and is better in 36 of 53 jurisdictions. A state can give part of its review budget to finding households that are being shorted; the gain is modest.

Keeping it working: an early simulation over FY2021–24 suggested that a ranker retrained only on the cases it flagged would lose about 15% of its effectiveness in four years. Rechecked without the leak (exp108), it doesn't: retraining only on flagged reviews did as well as retraining on every case (2.40× against 2.29× in 2024, mean of six states), so that finding is withdrawn. Retraining each year on the federal QC sample, which is random, fully reviewed and free, is still the simplest design.

2. FY2028 cost share

H.R. 1 (2025) makes states with payment error rates of 6% or more pay 5–15% of their SNAP benefit costs from FY2028.

All jurisdictions, per year	Ranker-guided reviews (floor on)	Random reviews	Reviewer FTEs
Low (1% of cases reviewed a month)	$0.21B avoided	$0.16B	about 770
Central (2% a month)	**$4.47B avoided**	**$1.03B**	about 1,540
High (3% a month)	$8.24B avoided	$2.39B	about 2,310

Examples:

- **Pennsylvania:** in the central scenario its rate falls from 9.21% to about 7.0%, halving its cost share (10% → 5%, about $209 million a year).
- **California:** from 10.93% to 8.50%, 15% → 10%, about $596 million a year.

What drives the range: mostly review volume and how long a found error would otherwise have lasted; the ranker's accuracy matters least. These figures show the size of the opportunity; they are not a forecast. `COST-SHARE-REPLICATE.md` reproduces the national and any single state's figure.

3. Synthetic data

Fit: after about 45 calibration experiments, the generator's error mix (which cases have errors, over vs under, which part of the case, why, how many dollars) matches the real data within about 2%, and the dollar error rate within about 1 point.

Transfer: models trained only on synthetic cases beat chance on real cases in every test, reaching about half to two-thirds of real-data performance.

Lessons:

- Realism mattered more than volume.
- Some training choices helped one kind of model and hurt others.
- A classifier can still tell fake cases from real ones, mainly by how household traits combine. The generator is good enough to teach, but not indistinguishable.

4. A small language model for QC review

Scores on 1,200 fresh real QC cases, under pass/fail rules committed before the cases were drawn. "Skill" is the score above lucky guessing:

Model	Overall	Skill above chance	Finds the error's location	Passes all checks?
Gemma 4 E4B, untrained	0.5%	0.5%	1%	—
Conventional classifier, same synthetic data	24%	15%	18%	—
Gemma 4 E4B, trained on synthetic cases	**33%**	**28%**	**24%** (bar 21%)	**Yes**

Gain over untrained: +33 points (95% interval +31 to +35), and it won on all five category questions. A half-size model trained the same way (E2B) scored 25% and missed the location bar.

Training method: supervised fine-tuning worked; reinforcement learning alone taught the model only to guess the most common answer.

Side effects: training didn't cost policy knowledge. On the CalFresh quiz it scored 74.4%, against 72.6% before training.

5. Flag cards

A flag card has three parts:

- why the ranker flagged the case;
- the parts of the case to check first;
- the rule that applies.

Measure	Result
Real error among the top three "check first" items	83% (Pennsylvania), 89% (New York), leak-free (exp108); a fixed list of the three most common locations: 80% and 79%
"N in 10" odds, before → after calibration	expected calibration error 0.12–0.22 → 0.05–0.08 (leak-free, exp108)
Small model (E2B) writing the card, automatic checks	97–99% pass
Rules line on 517 fresh cards vs the 31B model	100% precision and recall; every citation resolves
Notes grounded in rule excerpts (E4B)	cite the governing rule for 97% of checks; no invented citations

What works best: the model writes the plain note and code appends the rule line. Asking the model to put citations at the end made notes much harder to read.

Labelled cases needed: the where-to-look model beats generic advice after about 100 labelled error cases.

Still to do: a blind reviewer comparison of template vs model-written cards is packaged, waiting for reviewers.

6. Policy questions

Cornell CalFresh quiz, 500 questions	Accuracy
Gemma 4 E4B, no tools / best tool setup	72.6–73.4% / 77.0%
Gemma 4 31B, no tools (ours / Nava's)	84.0% / 85.8%
Gemma 4 31B with policy lookup, as of June 2026	**89.6%**

Cornell CalFresh quiz, 500 questions	Accuracy
Gemma 4 31B with lookup + topic router (pre-registered)	89.8% (router +1.6 over its own plain run)
GPT-6 Astra, no tools, untuned (Nava)	92.24%

What's in the library:

- 4,461 documents in Open Knowledge Format: federal regulations and statute, FNS guidance, California's regulations and 458 county letters, and other states' manuals;
- dates in force on every document;
- citation links between documents (2,672).

Organization mattered more than size. Adding old letters hurt search until they were made reachable only through citations. What remains wrong is mostly misapplied rules, not missing ones. Full results are in the companion summary.

What didn't work

Approaches that failed:

- **Reinforcement learning alone** for the QC model. It learned to guess the majority answer.
- **Asking the QC model to explain first.** Training it to list differences before answering made it call every case an error (overall 7%, against 33%).
- **Distilling the large model's tool use into the small one.** It learned the habit, but stopped answering. The cause was a training-data defect: the recording tool drops the final "submit" step.
- **Asking models how confident they are.** They were as confident when wrong as when right.
- **Automatic generator tuning.** It chased random noise and lost on fresh data.

What we learned from them:

- **Each lesson is recorded** in the experiment that produced it.

- **The repeated one:** check every result on fresh data under rules written down first. Several apparent gains (a tool recipe, generator auto-tuning, prompt changes) didn't survive that check.

Methodology notes

- **Data:**
  - **QC records:** federal SNAP QC public-use files, FY2020–2024 (about 167,000 cases). Each state set aside about a quarter of its cases for final testing (fixed in `exp68_holdouts.json`).
  - **Error rates:** USDA's official FY2025 rates.
  - **The quiz:** Cornell's CalFresh quiz is CC BY-NC-ND 4.0, used for evaluation only. It was never trained on, and no question text or per-question output was published.
- **Rigor:**
  - every experiment was recorded as hypothesis, method, result and verdict, failures included;
  - key tests were pre-registered, with pass/fail rules committed before running;
  - confidence intervals come from paired bootstrap (2,000 resamples);
  - real test cases were never used for training or tuning.
- **Compute:** one Mac Studio (M3 Ultra, 256 GB). Language models were trained with MLX and served with LM Studio. Every result here was produced on that machine.

Limitations

- **Public data only.** The models were built from federal public-use files, which omit detail a state's own case records hold. A state's data would likely do better, after a short adaptation step.
- **Scenarios, not forecasts.** The cost-share model leaves out:
  - timing relative to the federal sample month;

    - procedural errors outside the public file (published rates run 2–3 points above what the file implies);
    - behavior changes;
    - the delay option;
    - the full cost of reviews.
- **Small samples in small states.** Pooling helps, but forecasts are less reliable in small jurisdictions (within 20% of actual in 43 of 53 overall).
- **The language models are research results.** The QC model is well short of a human reviewer on the details of an error. The policy assistant was measured on one California multiple-choice benchmark, and many of its later changes were chosen after looking at that benchmark.
- **Rankers prioritize; they don't decide.** Every determination stays with a person.

Recommendations

1. **Pilot the ranker in one state** (Pennsylvania or New York), with the fairness floor on by default and the underpayment ranker as an optional second queue. Compare dollars found with the forecast.
2. **Refresh yearly** on the federal QC sample plus the pilot's own findings. Keep about 10% of pilot reviews on random cases to watch for blind spots.
3. **Attach flag cards in the pilot,** and run the packaged blind reviewer comparison (template vs model-written).
4. **Measure the two numbers that settle the cost-share range:** realistic review volume, and how long a found error would otherwise last.
5. **Keep the language models as helpers.**
    - **Policy assistant:** use it with citations shown, as a reference that points to the rule rather than an authority.
    - **Accuracy:** if accuracy above 92% matters, consult a stronger model only on the questions where local setups disagree.
    - **Next test:** build a held-out test of our own, with other states and written answers, before claiming further gains.

Technical reproduction

All code: github.com/ctrimm/snap-qc-per-experiments. Policy library and MCP server: github.com/ctrimm/state-policy-docs. Each experiment has a script and a write-up in `experiments/`, indexed in `README.md`.

Data

Download the FY2020–2024 SNAP QC public-use CSVs from snapqcdata.net/datafiles into `../snap-qc-data/raw/qcfy20XX_csv/` (a sibling of the repo). The folder names are what the code expects. Python 3 with numpy, joblib and scikit-learn 1.9.1; other scikit-learn versions may load or score the saved models differently.

Finding 1: the ranker for every state

```
# fit and package the context-aware pooled model for each jurisdiction (adoption rule
pre-registered in the script)
python3 experiments/exp82_context_all_states.py --workers 8 --package
# full held-out evaluation: top-10%/20% lift, $ per review hour, forecast check,
fairness floor
python3 experiments/exp83_context_models_full_eval.py --workers 6
# score a state's cases (pre-review fields only; see
deliverables/state_rankers/README.md for the columns)
python3 deliverables/state_rankers/score_cases.py --state PA --input cases.csv --output
ranked.csv --id-col CASE_ID
```

- **Expected (the defaults: leak-free inputs, official error label):** median top-20% lift 2.96×, top-10% 4.07×, forecasts within ±20% in 38 of 53 (`exp83_official_results.json`). `SNAPQC_LABEL=gap` reproduces exp111 (`exp83_gap_results.json`); `SNAPQC_LABEL=any` reproduces exp107 (`exp83_leakfree_results.json`); adding `SNAPQC_FEATURES=legacy` reproduces the published pre-exp106 figures.
- **Holdouts:** fixed per state in `exp68_holdouts.json` (25%, stratified, seed 68 + state code).

Finding 2: the cost-share figure

```
python3 experiments/cost_share_replicate.py             # national: $4.47B central (no
dependencies)
python3 experiments/cost_share_replicate.py --state PA   # one state, every step shown
# full rebuild from raw data: refits every ranker; stops unless every lift matches
exp83 exactly
python3 experiments/exp93_cost_share_current_models.py --workers 6
```

`COST-SHARE-REPLICATE.md` explains the model, the parameters for each scenario, and how to run it with any state's own inputs.

Finding 3: synthetic data

```
cd src && python -m synth.generate --n 20000 --seed 7 --dist ../experiments/dist.json --out ../data/synth.jsonl
```

`experiments/dist.json` holds the empirical distributions, so this runs without the raw data. The calibration and transfer checks are experiments 1–45 (see the index), and state-specific generators are in `src/synth/*_state.py`.

Finding 4: the QC language model

- **Model:** `mlx-community/gemma-4-e4b-it-bf16` with LoRA, in an mlx-lm virtualenv.
- **Training data:** synthetic cases rendered by `experiments/exp52_rlvr_dataset.py` and `exp52_sft_data.py`.

```
# supervised fine-tuning (16 layers, lr 1e-5, batch 4, 600 steps; checkpoint 400 was frozen for the confirmatory test)
experiments/exp52_sft_train.sh sft_lr1e-5_600 600 1e-5
# pre-registered confirmatory test on 1,200 fresh real cases: classifier, base and tuned, then the v1.2 gate
experiments/exp59_run.sh
```

- **Frozen adapter:** SHA-256 `a0bf35d7…5f5737e` (full hash in `exp59_confirmatory_gate.md`).
- **Fresh test items:** `eval_pilot/sample.py --seed 59 --exclude items.json`.
- **Expected:** composite 0.334 vs base 0.005 (+0.328 [+0.309, +0.347]), location 0.244 against a 0.210 bar.

Finding 5: flag cards

- **Card design and coverage:** exp62 and exp65.
- **Odds calibration:** exp74 (temperature scaling, T ≈ 2.9).
- **The Rules line at scale:** exp97, 517 fresh cards from Idaho, Utah and South Dakota.

Each write-up lists its script and inputs.

Finding 6: policy questions

The companion summary, *Grounding an Open Model in SNAP Policy* (`whitepaper/policy_qa_summary.pdf`), gives the environment and the commands for:

- the tools result (exp86 / exp98-3);
- the pre-registered router (exp105);
- the retrieval comparison (exp104).

The core run:

```
lms load google/gemma-4-31b --context-length 262144 --parallel 4 -y
export OPENAI_BASE_URL=http://localhost:1234/v1 OPENAI_API_KEY=lm-studio
cd experiments && inspect eval exp85_mcq_rag_task.py@calfresh_mcq_rag --model
openai/google/gemma-4-31b \
  -M responses_api=false -T mode=agent -T as_of=2026-06-01 --max-connections 4 --retry-
on-error 2
```

Project timeline

When	Milestone
Sept 28–29	Project started. Synthetic generator built and calibrated over about 45 experiments; first proof that synthetic-trained models predict real errors. Pennsylvania and New York pilots
Oct 2–4	Fairness analysis and the review floor. Pilots for Texas, South Dakota, Utah and Idaho. Language-model evaluation designed and run on untrained models
Oct 5–7	Supervised fine-tuning works where reinforcement learning failed; state rankers tuned, calibrated and rebuilt on pre-review data only
Oct 9	Pre-registered confirmation on 1,200 fresh real cases: the QC model passes all three checks
Oct 9–10	Flag cards designed and their odds calibrated; rankers for all 53 jurisdictions; first FY2028 cost-share scenarios; CalFresh quiz baseline
Oct 10–11	Pooled, state-context rankers beat state-only models everywhere (reported then as a median 4.0×, later found inflated); underpayment ranker; policy tools tested on small models

When	Milestone
Oct 11–12	Policy lookup lifts Gemma 4 31B on the CalFresh quiz (+4.8); grounded flag-card notes
Oct 15	Cost share re-run on the deployed rankers with the fairness floor on (reported then as $6.05 billion, later found inflated)
Oct 16–18	Flag-card Rules line at scale; `COST-SHARE-REPLICATE.md` for national and per-state reproduction; quiz audit; best policy setup 89.6%
Oct 19–23	Policy-assistant plateau: supersession notes, the date, 458 CDSS letters, citation links, a routing layer confirmed in a pre-registered run (89.8%)
Oct 26	Comparing with giannella/snap_qc exposed a feature leak in the rankers (exp106); leak-free rebuild (exp107): median 2.7×, $3.42 billion central cost share. Leak-free became the default. Same day: the official error definition became the default (exp110, exp111: median 2.6×, $3.15 billion on a label built from the benefit gap), then the label was corrected to the reviewer's finding (exp112): median 3.0×, $4.47 billion

Appendix: how concentrated are error dollars?

Why this table exists. The October 4 state pilot papers said, for example, that "the top tenth of errors by dollar size holds 83% of all error dollars" (South Dakota), 75% (Utah) and 74% (Idaho). Rechecked on October 23, those figures are reproducible, but they measure something else: the share held by the top tenth of **all reviewed cases**, where most cases have no error and count as $0 (column B). Measured over **error cases only** (column A), the top tenth holds 38–53% of error dollars (median 47%; national 47.8%). Column A is what the papers' wording describes, so the current state briefs use neither old figure. Both are here.

Data source: SNAP Quality Control public-use files, FY2020–2024, from snapqcdata.net/datafiles, as distributed with the USDA Food and Nutrition Service's QC technical documentation (one file per fiscal year, 167,002 reviewed cases in all).

Fields:

- `STATUS` (1 correct, 2 overissuance, 3 underissuance);

- `AMTERR` (the case's error amount in dollars, as the QC reviewer found it);
- `STATENAME` (the jurisdiction).

Definitions:

- **Unit:** one reviewed case, unweighted, with all five years pooled.
- **Error case:** `STATUS` 2 or 3.
- **Top 10%:** the largest `AMTERR` values, round(n × 10%) of them.
- **National:** all 53 jurisdictions pooled.
- **Median and 90th percentile:** the `AMTERR` of error cases.
- **Exclusions:** none.
- **Weights:** the QC sampling weights (`FYWGT`) are not applied, so these describe the QC sample, not the caseload.

Reproduce: `python3 whitepaper/concentration_table.py`. It reads the five files and rewrites this table.

How to read it: A says how much of the error money sits in the biggest tenth of errors. B mixes in how rare errors are: a state where most reviewed cases are correct gets a high B even when its errors are no more concentrated. For prioritizing reviews, A is the relevant measure, and it is why ranking by expected dollars beats ranking by the chance of an error.

Jurisdiction	Reviewed cases	Error cases	A: top 10% of error cases	B: top 10% of all cases	Median error	90th pct.
Alabama (AL)	4,163	1,424	46.0%	77.6%	$30	$177
Alaska (AK)	1,367	474	51.1%	79.7%	$35	$221
Arizona (AZ)	3,352	1,046	46.3%	82.3%	$35	$258
Arkansas (AR)	3,430	1,117	49.1%	78.1%	$23	$121
California (CA)	3,108	1,278	44.9%	73.7%	$29	$209
Colorado (CO)	3,595	1,271	49.3%	79.6%	$28	$184

Jurisdiction	Reviewed cases	Error cases	A: top 10% of error cases	B: top 10% of all cases	Median error	90th pct.
Connecticut (CT)	3,818	1,515	45.3%	74.5%	$29	$196
Delaware (DE)	1,652	633	44.3%	74.6%	$32	$224
District of Columbia (DC)	2,689	1,163	38.1%	64.6%	$56	$258
Florida (FL)	3,745	1,518	44.8%	72.7%	$29	$184
Georgia (GA)	3,902	1,987	45.6%	66.6%	$35	$230
Guam (GU)	732	343	41.4%	66.6%	$45	$308
Hawaii (HI)	2,009	884	49.6%	71.4%	$30	$168
Idaho (ID)	3,839	1,292	48.9%	74.9%	$17	$67
Illinois (IL)	3,246	1,140	40.4%	73.3%	$42	$226
Indiana (IN)	3,237	1,248	46.4%	75.8%	$30	$196
Iowa (IA)	3,892	1,297	50.8%	81.3%	$26	$183
Kansas (KS)	3,698	1,550	45.4%	67.7%	$22	$99
Kentucky (KY)	4,073	1,219	52.3%	83.5%	$21	$157
Louisiana (LA)	3,325	1,411	49.8%	70.8%	$27	$140
Maine (ME)	3,434	1,423	44.7%	73.4%	$26	$176
Maryland (MD)	2,279	872	44.7%	74.1%	$41	$230
Massachusetts (MA)	3,483	1,287	46.5%	78.6%	$28	$229
Michigan (MI)	3,460	1,511	45.0%	70.0%	$32	$201

Jurisdiction	Reviewed cases	Error cases	A: top 10% of error cases	B: top 10% of all cases	Median error	90th pct.
Minnesota (MN)	4,077	1,566	43.9%	73.9%	$27	$167
Mississippi (MS)	4,158	1,547	46.5%	73.8%	$31	$176
Missouri (MO)	2,933	1,174	48.0%	72.5%	$23	$123
Montana (MT)	2,421	1,008	49.0%	74.3%	$23	$165
Nebraska (NE)	3,738	1,307	49.7%	77.0%	$20	$137
Nevada (NV)	4,047	1,351	49.4%	77.2%	$25	$156
New Hampshire (NH)	2,328	782	44.0%	77.1%	$28	$162
New Jersey (NJ)	2,327	756	49.2%	79.7%	$22	$137
New Mexico (NM)	3,845	1,838	48.9%	73.0%	$25	$202
New York (NY)	3,257	1,194	46.2%	78.6%	$30	$230
North Carolina (NC)	3,983	1,435	45.9%	78.8%	$29	$207
North Dakota (ND)	1,691	645	49.0%	77.3%	$24	$170
Ohio (OH)	3,475	1,317	50.8%	77.6%	$21	$143
Oklahoma (OK)	3,829	1,562	47.9%	73.0%	$30	$169
Oregon (OR)	3,037	1,207	45.0%	74.0%	$33	$230

Jurisdiction	Reviewed cases	Error cases	A: top 10% of error cases	B: top 10% of all cases	Median error	90th pct.
Pennsylvania (PA)	3,268	1,395	50.7%	74.0%	$26	$169
Rhode Island (RI)	3,435	1,646	40.4%	64.0%	$32	$196
South Carolina (SC)	3,737	1,355	44.7%	74.6%	$29	$166
South Dakota (SD)	2,365	607	50.4%	82.5%	$18	$81
Tennessee (TN)	3,807	1,348	45.9%	75.7%	$26	$148
Texas (TX)	3,457	1,530	47.1%	69.5%	$29	$155
Utah (UT)	4,101	1,347	50.2%	76.0%	$21	$106
Vermont (VT)	2,720	939	52.6%	78.0%	$19	$111
Virgin Islands (VI)	998	420	49.7%	77.1%	$28	$229
Virginia (VA)	2,942	1,366	45.7%	67.1%	$32	$167
Washington (WA)	3,060	805	45.5%	84.5%	$32	$213
West Virginia (WV)	3,268	1,226	46.7%	73.7%	$24	$134
Wisconsin (WI)	3,979	1,305	49.3%	78.4%	$24	$144
Wyoming (WY)	1,191	405	46.2%	74.0%	$21	$97
National	167,002	63,286	47.8%	76.2%	$28	$178